

Google Colab all codes:

<https://colab.research.google.com/drive/1HIhnpRhZZns70xktl00sVkUK2VsNW0Of?usp=sharing>

Customer Relationship & Change Management



Name: Ryan Rahman

StudentID: w1967351

Module leader: Panagiotis Chountas

Course code: 6BUI017W1

Task 1 - Data Loading

1a) This code satisfies Task 1a and uses Python to retrieve the list of S&P 500 tickers from the Wikipedia website. The code first uses import statements to include all the required libraries, including "numpy," "pandas," "pandas_datareader,"

```
import numpy as np
import pandas as pd
import pandas_datareader as dr
```

```
#define URL
import requests
sp500_url = 'https://en.wikipedia.org/wiki/List_of_S%26P_500_companies'
header = {
    "User-Agent": "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/50.2661.75 Safari/537.36",
}
r = requests.get(sp500_url, headers=header)

data_table=pd.read_html(r.text)[0]

/tmp/ipython-input-3159576941.py:21: FutureWarning: Passing literal html to 'read_html' is deprecated and will be removed in a future version. To read from a literal string,
data_table=pd.read_html(r.text)[0]
```

and "requests," which has been used to download contents from the Wikipedia webpage. The code then defines the URL of the Wikipedia webpage that holds information about all of the firms in the S&P 500 and downloads the webpage's HTML contents by calling requests.get(). The most crucial part of successfully implementing Task 1a is accomplished by applying pd.read_html(r.text) to scan the webpage and import its first table, which holds information such as tickers and others, into a pandas dataframe titled "data_table."

In this part of Task 1a, the list of companies within the S&P 500 Index was collected successfully from the Wikipedia webpage using the requests library by saving the webpage and then analysing the HTML content with pandas.read_html().

data_table

	Symbol	Security	GICS Sector	GICS Sub-Industry	Headquarters Location	Date added	CIK	Founded
0	MMM	3M	Industrials	Industrial Conglomerates	Saint Paul, Minnesota	1957-03-04	66740	1902
1	AOS	A. O. Smith	Industrials	Building Products	Milwaukee, Wisconsin	2017-07-26	91142	1916
2	ABT	Abbott Laboratories	Health Care	Health Care Equipment	North Chicago, Illinois	1957-03-04	1800	1888
3	ABBV	AbbVie	Health Care	Biotechnology	North Chicago, Illinois	2012-12-31	1551152	2013 (1888)
4	ACN	Accenture	Information Technology	IT Consulting & Other Services	Dublin, Ireland	2011-07-06	1467373	1989
...	...	...	...	...	...	...	...	...
498	XYL	Xylem Inc.	Industrials	Industrial Machinery & Supplies & Components	White Plains, New York	2011-11-01	1524472	2011
499	YUM	Yum! Brands	Consumer Discretionary	Restaurants	Louisville, Kentucky	1997-10-06	1041061	1997
500	ZBRA	Zebra Technologies	Information Technology	Electronic Equipment & Instruments	Lincolnshire, Illinois	2019-12-23	877212	1969
501	ZBH	Zimmer Biomet	Health Care	Health Care Equipment	Warsaw, Indiana	2001-08-07	1136869	1927
502	ZTS	Zoetis	Health Care	Pharmaceuticals	Parsippany, New Jersey	2013-06-21	1555280	1952

503 rows x 8 columns

This generated an organised DataFrame consisting of 503 rows and 8 columns, containing key information for each company such as tickers, names, GICS sectors and sub-industries, headquarters, dates of addition to the index, CIK codes, and year of founding. The list of companies is the whole and accurate makeup of the S&P 500 Index, giving the required tickers essential for pulling historic stock data in the proceeding task.

This code line will extract the "Symbol" column of the data_table dataframe, which holds the tickers of the S&P 500 stocks. This will isolate the tickers only so they can be used at a later point in time to extract stock information.

data_table["Symbol"]

```
Symbol
0    MMM
1    AOS
2    ABT
3    ABBV
4    ACN
...    ...
498  XYL
499  YUM
500  ZBRA
501  ZBH
502  ZTS
```

1b) In this code, it downloads the daily stock prices of all S&P 500 companies from Yahoo Finance using the 'yfinance' library from 01-01-2022 to 01-01-2025. The ticker symbols were taken from

`data_table['Symbol']`. The historical data of these stocks is downloaded and stored in a Pandas DataFrame named 'Data_stock_price.' The extracted data can be utilised to calculate financial ratios such as returns, beta, and volatility. Many ticker symbols were removed effectively; however, some symbols (BF.B, SOLS, Q, and BRK.B) weren't withdrawn successfully, maybe because they were being delisted or lacking timezone information.

```
import yfinance as yf
!pip install yahoofinancials
from yahoofinancials import YahooFinancials
from datetime import datetime
#Define a start date and end date
start = '2022-01-01'
end = '2025-01-01'
##Read Stock price Data (EG Apple stock price)
Data_stock_price = yf.download(data_table['Symbol'].values.tolist(), start= start, end= end)
```

```
Collecting yahoofinancials
  Downloading yahoofinancials-1.20.tar.gz (51 kB)
    51.9/51.9 kB 2.5 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.12/dist-packages (from yahoofinancials) (2025.2)
Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.12/dist-packages (from yahoofinancials) (2.32.4)
Collecting appdirs>=1.4.4 (from yahoofinancials)
  Downloading appdirs-1.4.4-py2.py3-none-any.whl.metadata (9.0 kB)
Requirement already satisfied: frozendict>=2.3.4 in /usr/local/lib/python3.12/dist-packages (from yahoofinancials) (2.4.6)
Requirement already satisfied: peewee>=3.16.2 in /usr/local/lib/python3.12/dist-packages (from yahoofinancials) (3.18.3)
Requirement already satisfied: BeautifulSoup4>=4.11.1 in /usr/local/lib/python3.12/dist-packages (from yahoofinancials) (4.13.5)
Requirement already satisfied: lxml>=4.9.1 in /usr/local/lib/python3.12/dist-packages (from yahoofinancials) (5.4.0)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from BeautifulSoup4>=4.11.1->yahoofinancials) (2.8)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from BeautifulSoup4>=4.11.1->yahoofinancials) (4.15.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yahoofinancials) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yahoofinancials) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yahoofinancials) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yahoofinancials) (2025.10.5)
Downloading appdirs-1.4.4-py2.py3-none-any.whl (9.6 kB)
Building wheels for collected packages: yahoofinancials
  Building wheel for yahoofinancials (setup.py) ... done
  Created wheel for yahoofinancials: filename=yahoofinancials-1.20-py3-none-any.whl size=38618 sha256=54d3b4f743d44df0a8ae9692d2cbf68d4bae85635a1e5f5bbbc564626d
  Stored in directory: /root/.cache/pip/wheels/b0/e1/ca/e683b02e57db550881c8ebb89ba3eccb7a5cbebfad7f03acea
Successfully built yahoofinancials
Installing collected packages: appdirs, yahoofinancials
```

1c) The downloaded stock price data was cleaned in this step to be used accurately in financial calculations. The stocks whose price histories were missing entirely were dropped using the `dropna(axis=1, how='all')` because their absence would interfere with the computation of return and beta. In the stocks that were not entirely missing their price histories, missing data points that often occurred due to trading stops and missing entries were handled through forward fill (`ffill`). The first few rows of the cleaned data were displayed using the `head()` function to confirm the cleaning of the data. The data had been made complete and consistent in this step to allow accurate computation.

```
import yfinance as yf
import pandas as pd
import requests

# 1. Get S&P 500 tickers from Wikipedia
# -----
sp500_url = 'https://en.wikipedia.org/wiki/List_of_S&P_500_companies'
header = {"User-Agent": "Mozilla/5.0"}
r = requests.get(sp500_url, headers=header)

tables = pd.read_html(r.text)
data_table = None
for table in tables:
    if 'Symbol' in table.columns:
        data_table = table
        break

# 2. Fix ticker symbols for Yahoo Finance
# -----
tickers = data_table["Symbol"].str.replace(".", "-", regex=False).tolist()

# Remove invalid/delisted tickers
invalid_tickers = ['SOLS', 'Q']
tickers = [t for t in tickers if t not in invalid_tickers]

# 3. Download stock price data
# -----
start = '2022-01-01'
end = '2025-01-01'

Data_stock_price = yf.download(tickers, start=start, end=end)

Data_stock_price_clean = Data_stock_price.dropna(axis=1, how='all')
Data_stock_price_clean = Data_stock_price_clean.fillna(method='ffill', axis=1)
Data_stock_price_clean.head()
```

Output of 1c:

```
/tmp/ipython-input-2906227424.py:12: FutureWarning: Passing literal html to 'read_html' is deprecated and will be removed in a future version. To read from a literal string, wrap it in a 'StringIO' object.
tables = pd.read_html(r.text)
/tmp/ipython-input-2906227424.py:34: FutureWarning: YF.download() has changed argument auto_adjust default to True
Data_stock_price = yf.download(tickers, start=start, end=end)
[*****100%*****] 501 of 501 completedDownloaded stock data for 501 tickers
```

Price	Close																	... Volume									
Ticker	A	AAPL	ABBV	ABNB	ABT	ACGL	ACN	ADBE	ADI	ADM	...	NY	WYNN	XEL	XOM	XYL	XYZ	YUM	ZBH	ZBRA	ZTS						
Date																											
2022-01-03	152	320023	178.270309	116.779297	172.679993	128.996140	42.362530	383.711884	564.369995	165.624512	60.489777	...	3831100	2437800	3501100	24282400	759100	7315700	1251400	1184809	272600	2772700					
2022-01-04	147	170685	176.007782	116.555084	170.800003	125.962372	42.914051	380.969910	554.000000	164.128632	61.612446	...	3089700	2292300	4197000	38584000	925400	14768500	935900	1400800	346000	4664000					
2022-01-05	144	649536	171.325974	117.167343	162.250000	125.396393	42.410072	374.260712	514.429993	161.594925	61.131298	...	3737600	3439900	4166000	34033300	1090200	17546200	977900	1895715	403700	4749400					
2022-01-06	145	155685	168.465973	116.615440	159.750000	125.377823	42.657307	356.187531	514.119995	162.109161	61.674824	...	3315200	2583200	2296000	30668500	703400	16244200	862400	1088813	338300	3103400					
2022-01-07	141	291245	168.632507	116.313614	166.050003	125.767525	42.856995	349.356865	510.700012	157.855209	62.200508	...	3309900	1720400	2673100	23985400	765000	9426000	833700	1690230	432800	2206500					

5 rows * 2505 columns

Task 2 - Define and calculate in Google Colab the Daily Return, Beta and Annual Volatility metrics

2a) In this section of the code, daily returns and annual volatility for all S&P 500 stocks have been calculated based on clean data from stock prices.

The formula used to calculate returns is the percentage change in the closing stock prices from one day to another to determine how stocks have performed from one day to another. The formula to calculate the `annual\_volatility` will then be implemented by finding the standard deviation of returns and then multiplying it by the square root of 252 to determine yearly volatility. The result shows that there is a table of returns for all stocks pertaining to returns in the first five days and their individual `annual\_volatility` values. A stock whose `volatility` values measure high represents high risks and fluctuations in performance relative to stocks whose values measure lower and represent lower risks. These results play critical roles in informing CRM strategies and decisions for diversified stocks to be selected depending on their risks and volatilities identified from their data.

```
import math
daily_returns = Data_stock_price_clean['Close'].pct_change().dropna()
annual_volatility = daily_returns.std() * math.sqrt(252)
daily_returns.head(), annual_volatility.head()
```

```
*** (Ticker      A      AAPL      ABBV      ABNB      ABT      ACGL \
Date
2024-03-28 -0.012621 -0.010559  0.009703 -0.008713  0.001586  0.010494
2024-04-01  0.001969 -0.008456 -0.007359 -0.009639 -0.013813 -0.001406
2024-04-02 -0.009549 -0.006999 -0.000387 -0.019955 -0.000625 -0.008883
2024-04-03 -0.003052  0.004797 -0.018595 -0.004089 -0.006006  0.008088
2024-04-04 -0.016628 -0.004892 -0.053178 -0.003138 -0.010514 -0.012361

Ticker      ACN      ADBE      ADI      ADM      ...      WY      WYNN \
Date
2024-03-28  0.016630  0.000397  0.023069 -0.002858  ... -0.000557  0.004323
2024-04-01 -0.021465 -0.004974 -0.005915 -0.007324  ... -0.009746  0.042356
2024-04-02 -0.007990 -0.005736 -0.008849  0.005613  ... -0.003656 -0.012200
2024-04-03 -0.012424 -0.004407  0.002463 -0.000797  ... -0.013830  0.011020
2024-04-04 -0.005447 -0.019919 -0.017455  0.009417  ... -0.008586 -0.002819

Ticker      XEL      XOM      XYL      XYZ      YUM      ZBH \
Date
2024-03-28  0.006931  0.011046 -0.001082  0.016831  0.009685 -0.004751
2024-04-01 -0.008744  0.006452 -0.004643 -0.036888  0.013127 -0.016897
2024-04-02  0.015015  0.019574 -0.004042 -0.027621  0.001708 -0.009865
2024-04-03  0.000185  0.000168  0.005776  0.005555 -0.012082  0.011987
2024-04-04 -0.005916  0.003520 -0.012494 -0.061770 -0.019639 -0.018845

Ticker      ZBRA      ZTS
Date
2024-03-28  0.007621  0.004154
2024-04-01 -0.001460 -0.012942
2024-04-02 -0.014950 -0.012035
2024-04-03 -0.012648 -0.012363
2024-04-04  0.000717  0.015647

[5 rows x 501 columns],
Ticker
A      0.263067
AAPL   0.230234
ABBV   0.259462
ABNB   0.333592
ABT    0.179501
dtype: float64)
```

2b) This code calculates Beta values for all stocks in S&P 500 relative to S&P 500 index (^GSPC) from 01-01-2022 to 01-01-2025. The calculate_betas function begins by downloading market history and computing market returns and standard deviation. For each stock in the market index, download history for each stock, compute their returns, then compute their correlation coefficient to market returns. The Beta formula, Correlation Coefficient × (Stock returns standard deviation/Market returns standard deviation), allows Beta to be processed for each stock. In the resulting DataFrame 'Beta', columns represent Stock_Name and Beta. The .head() function provides the first few Beta values to quickly verify Beta computation accuracy.

```
import yfinance as yf
import pandas as pd

# --- 2b. Beta Calculation ---

def calculate_betas(market_ticker, stock_list, start_date, end_date):
    # Download market data
    market = yf.download(market_ticker, start=start_date, end=end_date)
    market['daily_return'] = market['Close'].pct_change()
    market.dropna(inplace=True)
    market_std = market['daily_return'].std()

    betas_list = []

    # Loop through each stock
    for stock in stock_list:
        data = yf.download(stock, start=start_date, end=end_date)
        data['daily_return'] = data['Close'].pct_change()
        data.dropna(inplace=True)

        # Correlation with market
        corr = data['daily_return'].corr(market['daily_return'])
        # Beta formula
        beta_val = corr * (data['daily_return'].std() / market_std)
        betas_list.append(beta_val)

    # Create a DataFrame for results
    beta_df = pd.DataFrame({
        'Stock_Name': stock_list,
        'Beta': betas_list
    })

    return beta_df

# Call the function for all S&P 500 stocks vs S&P 500 index (^GSPC)
betas_df = calculate_betas('^GSPC', data_table['Symbol'].values.tolist(), '2022-01-01', '2025-01-01')

# Quick check
betas_df.head()
```

Output of 2b:

[illegible]

2c) The code determines the stock's daily returns through the calculation of the percentage change of the 'Close' stock price using the code `daily_returns = Data_stock_price['Close'].pct_change()`. This shows the difference in the stock's price from day to day. The code also determines the

```
import math
daily_returns = Data_stock_price['Close'].pct_change()
annual_volatility = daily_returns.std() * math.sqrt(252)
```

annual volatility

annual stock volatility through the calculation of the standard deviation of the stock's daily returns and the square-root of 252 trading days in a year using the code `annual_volatility = daily_returns.std() * math.sqrt(252)`. This code generates one value of the stock's volatility per year because the stock data from the 'Close' column has been used.

Output of 2c:

```

Ticker
A      0.296259
AAPL   0.270935
ABBV   0.219958
ABNB   0.467875
ABT    0.217839
...
XYZ    0.653825
YUM    0.191335
ZBH    0.245157
ZBRA   0.400050
ZTS    0.275016
501 rows x 1 columns
dtype: float64

```

Task 3 - Stock Segmentation with Agglomerative Clustering

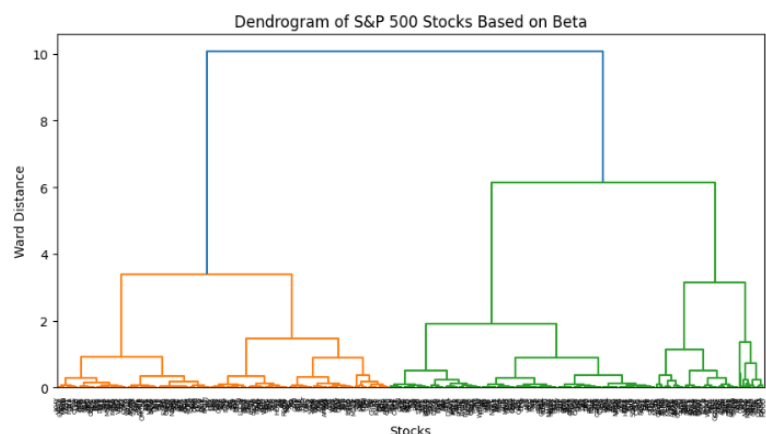
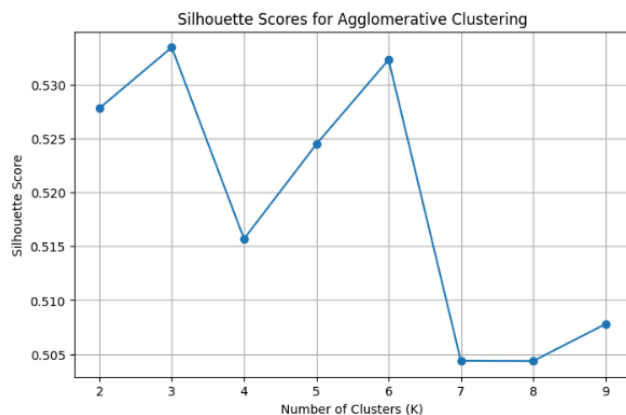
3a) Agglomerative clustering is a method for systematically grouping stocks. Each stock first forms a separate group, and we then continue to gradually combine the stocks that are most similar. This continues until every data point is a member of a single large group. This may be shown as a dendrogram, which is a tree diagram.

Since we don't need to know how many categories to create in advance, this strategy works well for classifying stocks. Based on their beta ratings, which indicate their risk and market sensitivity, it demonstrates how stocks naturally cluster together. By displaying the locations of the largest gaps between clusters, the dendrogram also assists us in determining the ideal number of groups.

3b) To determine the optimal number of clusters, the silhouette value was employed. The silhouette value allows the evaluation of how well each stock fits in its particular group against the quality of the clustering in the remaining groups. A greater value of the silhouette suggests a clear distinction of the groups. Various choices of K were investigated in the study based on the average value of the silhouette. This step will ensure alignment of the clusters both within themselves and different from each other, meaning that the stocks will be similar within their groups according to their beta value while being diverse from stocks in the other groups. The value of K which received the highest score from the results of the silhouette method was chosen—we will select the value of K to be 4 if it offers the best clusters. Additionally, a dendrogram was plotted to visualize the hierarchical grouping of stocks, helping to confirm the optimal number of clusters and observe how stocks naturally merge based on their Beta values.

Output for 3b:

K=2 → Silhouette Score = 0.5278
K=3 → Silhouette Score = 0.5334
K=4 → Silhouette Score = 0.5157
K=5 → Silhouette Score = 0.5245
K=6 → Silhouette Score = 0.5323
K=7 → Silhouette Score = 0.5044
K=8 → Silhouette Score = 0.5044
K=9 → Silhouette Score = 0.5078



3c)

In Task 3c, the technique of agglomerative clustering was used to assess the impact of various values of K on the grouping of stocks based on their Beta values from the S&P 500 stocks. Also, the Ward method of linkage was used to obtain the groups of stocks for the selected value of K of 3, 4, 5, and 6. The results showed the level of sophistication of the grouping of stocks as the value of K increases. The grouping at K = 3 includes many stocks in a large group of low-beta stocks and two groups of higher-beta stocks. Raising the value of K from 3 to 4 and then to 5 refines the groupings of stocks considerably, and the groups of stocks containing stocks of very high beta break into smaller and specialized groups. These groups comprise only 18 stocks when the value of K is 5. Finally, when the value of K increases to 6, the groupings are no longer clear. The value of K that balances the groupings of stocks properly is 4 because it also identifies the defensive stocks, moderate, cyclical, and stocks of the high-risk category. The final step has resulted in identifying the group each stock belongs to through its sensitivity in the market. Although the evaluation of the clusters in Task 3b was carried out from k=2 to k=9, only the visual representation of the clusters from k=3 to k=6 has been illustrated below. The reason behind this being left out is that the visual representation of the clusters becomes confusing when the value of k increases substantially, and k=3–6 helps create readable stock separation.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score

# 1. Load the clean betas dataframe
clean_betas_df = betas_df.dropna(subset=['Beta']).copy()

# Extract Beta values
X = clean_betas_df[['Beta']].values

# 2. Test range of clusters for silhouette score (optional)
range_n_clusters = range(2, 10)
silhouette_scores = []

for n_clusters in range_n_clusters:
    model = AgglomerativeClustering(n_clusters=n_clusters, linkage='ward')
    labels = model.fit_predict(X)
    score = silhouette_score(X, labels)
    silhouette_scores.append(score)
    print(f"K={n_clusters} -> Silhouette Score = {score:.4f}")

# Plot silhouette scores
plt.figure(figsize=(8,5))
plt.plot(range_n_clusters, silhouette_scores, marker='o')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Silhouette Score')
plt.title('Silhouette Scores for Agglomerative Clustering')
plt.grid(True)
plt.show()

# 3. Plot clustering results for K = 3 to 6 and show counts
for K in [3, 4, 5, 6]:
    model = AgglomerativeClustering(n_clusters=K, linkage='ward')
    labels = model.fit_predict(X)
    clean_betas_df['Cluster'] = labels

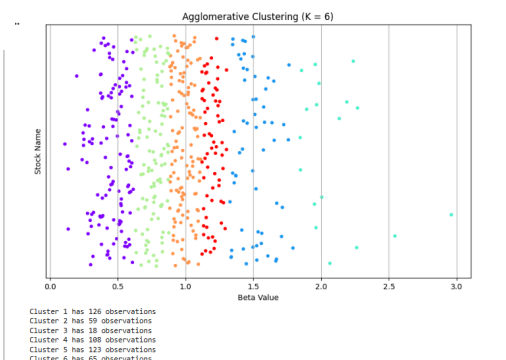
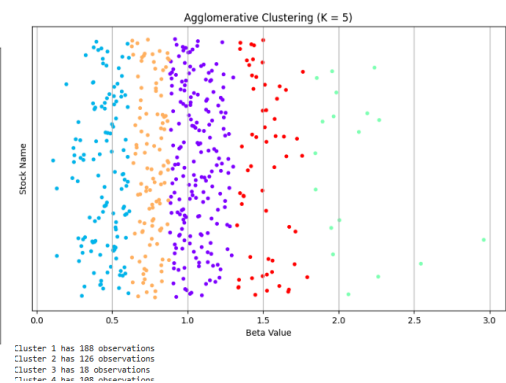
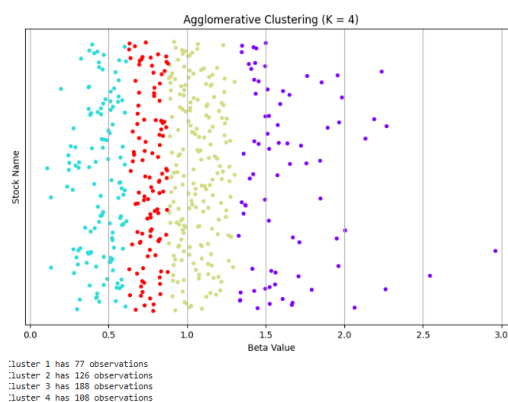
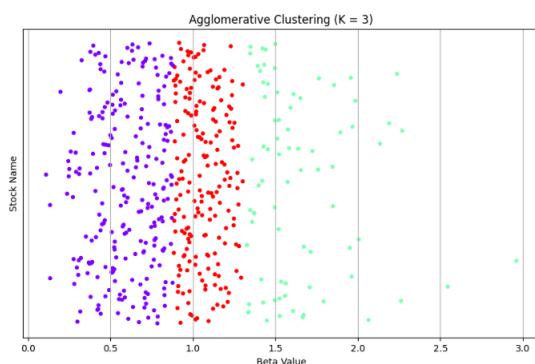
    plt.figure(figsize=(10,6))
    plt.scatter(clean_betas_df['Beta'], clean_betas_df['Stock Name'],
                c=labels, cmap='rainbow', s=12)
    plt.xlabel("Beta Value")
    plt.ylabel("Stock Name")
    plt.title(f'Agglomerative Clustering (K = {K})')
    plt.yticks([]) # hides stock names for clarity
    plt.grid(True)
    plt.show()

# Show cluster counts
cluster_counts = np.bincount(labels)
for i, count in enumerate(cluster_counts):
    print(f"Cluster {i+1} has {count} observations")
print("\n")

# 4. Display final clustered table for the best K
best_K = 4 # pick the best K from silhouette scores
final_model = AgglomerativeClustering(n_clusters=best_K, linkage='ward')
clean_betas_df['Cluster'] = final_model.fit_predict(X)

clean_betas_df.head()
```

Output of 3c:



Task 4 - Beta and Annual Volatility Metrics

4a)

K-Means Classification is an effective way of grouping stocks on the basis of Beta and Annual Volatility because it helps in identifying groups of stocks that are closely related in terms of risk. K-Means Classification also helps in effectively distinguishing between groups of low-risk stocks, moderate-risk stocks, and high-risk stocks.

K-Means is effective in scenarios where the approximate value for the number of clusters (K) is known. K-Means is also efficient for larger-sized datasets. K-Means clusters are simple to understand. This allows the investor to make effective portfolio diversification and risk management decisions. However, K-Means requires spherical clusters. This might affect the analysis performed on the data. K-Means also requires all clusters to be of equal size.

4b)

Several conventional methods have been used to arrive at the optimal level of K for the K-Means clustering model. Firstly, the Elbow Point Method was employed, whereby the graph of the Within-Cluster Sum of Squares (WCSS) for various numbers of clusters (various levels of K) was plotted to locate the stage beyond which further addition of clusters did not lead to a significant further reduction in the WCSS. Furthermore, the Silhouette Score of the stocks for each level of clusters K has been determined to evaluate the degree to which the stocks belonging to each cluster are very alike to each other and different from the stocks of similar clusters, the Silhouette Score increasing with the degree of separation and compactness of the clusters.

Moreover, the objective of selecting the optimal level of clusters for business understanding and interpretability has also been ensured, such that a distinction could be successfully drawn between low-risk defensive stocks and high beta stocks. Furthermore, the graphical confirmation of Beta vs. Volatility plots at various levels of K has also enabled the confirmation of the result.

4c) During this step, the stock portfolio will be segmented using the KMeans algorithm based on the two important financial factors: Beta and Annual Volatility. The code begins with the computation of the return series using the cleaned stock price series and then determines the annual volatility of each stock by multiplying the standard deviation of the return series with the square root of the number of trading days in a year (which is taken as the square root of 252). The Beta of each stock will be calculated against the S&P 500 stock index (^GSPC) as the covariance of the stock return series and market return series divided by the variance of the market return series. All the factors will be combined together in a dataframe to capture the risk characteristics of each stock. The dataframe is preprocessed using StandardScaler before applying the KMeans technique to form groups of stocks according to their level of risk. The resulting clusters group stocks with similar Beta and Annual Volatility, helping to identify low-,

medium-, and high-risk stock categories for portfolio management. The results are visualised using scatter and violin plots to show the distribution of Beta and Annual Volatility across the clusters.

Code of 4c:

```
import yfinance as yf
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import requests

# --- 1. Ensure cleaned stock price data is available ---
# The following code is added to ensure Data_stock_price_clean is defined.

# Get S&P 500 tickers from Wikipedia
sp500_url = 'https://en.wikipedia.org/wiki/List_of_S&P_500_companies'
header = {'User-Agent': 'Mozilla/5.0'}
r = requests.get(sp500_url, headers=header)

tables = pd.read_html(r.text)
data_table = None
for table in tables:
    if 'Symbol' in table.columns:
        data_table = table
        break

# Fix ticker symbols for Yahoo Finance
tickers = data_table['Symbol'].str.replace(".", "-", regex=False).tolist()

# Remove invalid/delisted tickers (as identified in previous runs)
invalid_tickers = ['SOLS', 'Q']
tickers = [t for t in tickers if t not in invalid_tickers]

# Download stock price data
start = '2022-01-01'
end = '2025-01-01'
Data_stock_price = yf.download(tickers, start=start, end=end)

# Clean Data_stock_price (from cell 758GuoX2fz5o)
Data_stock_price_clean = Data_stock_price.dropna(axis=1, how='all')
Data_stock_price_clean = Data_stock_price_clean.fillna(method='ffill')

# --- 2. Compute daily returns ---
daily_returns = Data_stock_price_clean['Close'].pct_change().dropna()

# --- 3. Compute annual volatility ---
annual_volatility = daily_returns.std() * np.sqrt(252)

# --- 4. Compute Beta relative to S&P 500 ('GSPC') ---
market_data = yf.download('GSPC', start='2022-01-01', end='2025-01-01')['Close']
market_returns = market_data.pct_change().dropna()

beta_values = {}
for stock in daily_returns.columns:
    # Ensure market_returns and daily_returns[stock] are aligned by index
    common_index = daily_returns[stock].index.intersection(market_returns.index)
    if len(common_index) > 1:
        # Ensure both inputs to np.cov are 1-D arrays (Series)
        stock_returns_aligned = daily_returns[stock].loc[common_index]
        market_returns_aligned = market_returns.loc[common_index]['GSPC'] # Select the Series from DataFrame
        covariance = np.cov(stock_returns_aligned, market_returns_aligned)[0, 1]
        beta = covariance / market_returns_aligned.var()
        beta_values[stock] = beta
    else:
        beta_values[stock] = np.nan # Assign NaN if not enough common data

# --- 5. Combine metrics into a DataFrame ---
stock_metrics = pd.DataFrame({
    'Beta': pd.Series(beta_values),
    'Annual_Volatility': annual_volatility
}).dropna()

# --- 6. Standardize features for K-Means ---
scaler = StandardScaler()
stock_metrics_scaled = scaler.fit_transform(stock_metrics)

# --- 7. Determine K (optional: can use Elbow/Silhouette elsewhere) ---
# For simplicity, let's assume K=4 (you can adjust after plotting elbow/silhouette)
K = 4

# --- 8. Apply K-Means Clustering ---
kmeans = KMeans(n_clusters=K, random_state=42, n_init=10)
stock_metrics['Cluster'] = kmeans.fit_predict(stock_metrics_scaled)

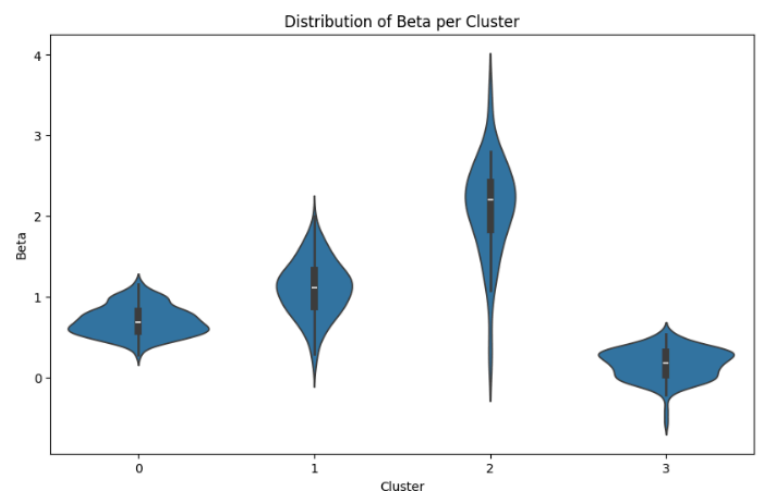
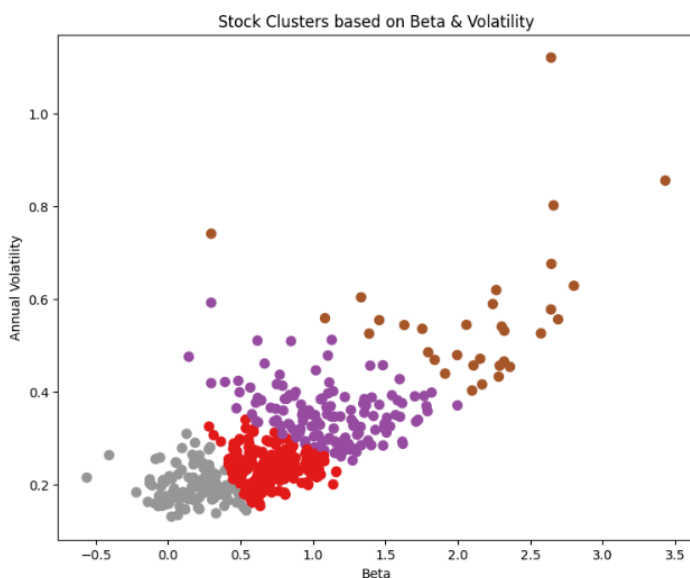
# --- 9. Analyze cluster profiles ---
cluster_summary = stock_metrics.groupby('Cluster').agg({
    'Beta': ['mean', 'min', 'max'],
    'Annual_Volatility': ['mean', 'min', 'max'],
    'Cluster': 'count'
}).rename(columns={'Cluster': 'Count'})
print("Cluster Profiles (Beta & Annual Volatility):")
print(cluster_summary)

# --- 10. Optional: Visualize clusters ---
plt.figure(figsize=(11, 7))
plt.scatter(stock_metrics['Beta'], stock_metrics['Annual_Volatility'],
            c=stock_metrics['Cluster'], cmap='Set1', s=50)
plt.xlabel('Beta')
plt.ylabel('Annual Volatility')

plt.figure(figsize=(10, 6))
sns.violinplot(x='Cluster', y='Beta', data=stock_metrics)
plt.title('Distribution of Beta per Cluster')
plt.xlabel('Cluster')
plt.ylabel('Beta')
plt.show()

plt.title('Stock Clusters based on Beta & Volatility')
plt.colorbar(label='Cluster')
plt.show()
```

Output of 4c:



Task 5 - Review of Results

5a) Daily Return, Beta, and Annual Volatility are equally crucial in allowing the investor to create a diversified portfolio from the derived clusters. Daily Return provides information about the short-term performance of each stock. An investor can identify the growth pattern of the stock through Daily Return and understand whether the stock will steadily grow in value or keep fluctuating. Beta value determines the stock's performance relative to the overall market. Beta stock value will enable the investor to identify defensive stocks and stocks that are sensitive in the market. A stock's defensive nature will be shown by a stock of low Beta value, and stocks actively involved in the market will be represented by stocks of high Beta value. The Annual Volatility value determines the stock's performance regarding its overall risk level. The stock's price change during the year will be shown through the stock's annual volatility value. By incorporating the above three factors in the derived clusters, the investor can evaluate the groups of stocks that collectively display the same type of performance.

5b) During this step, the results of the KMeans clustering are reviewed to understand the characteristics and business value of the stock segments for investors. The clusters are analyzed in terms of Beta and Annual Volatility, providing insight into the risk profiles of the stocks. Low-risk stocks tend to have lower Beta and volatility, while high-risk stocks show higher values for both metrics, allowing investors to identify groups suitable for different portfolio strategies. The scatter plot visually displays the segmentation, showing how stocks are grouped according to their risk characteristics, while the boxplots summarize the distribution of Beta and Annual Volatility within each cluster, highlighting variations and central tendencies. These visualizations help investors quickly assess the diversity and stability of each cluster, supporting informed portfolio construction and risk management decisions.

Code of 5b:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming 'stock_metrics' contains Beta and Annual Volatility
# and 'kmeans_labels' contains the cluster labels from KMeans

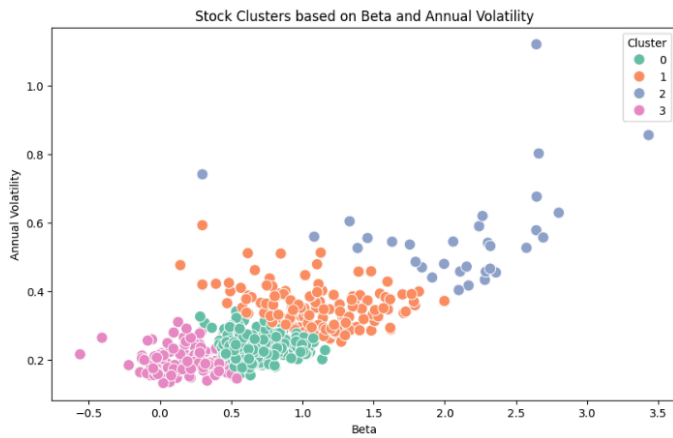
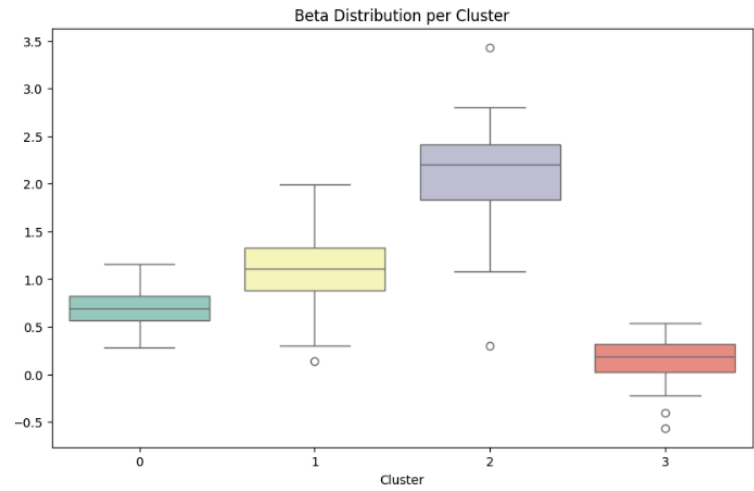
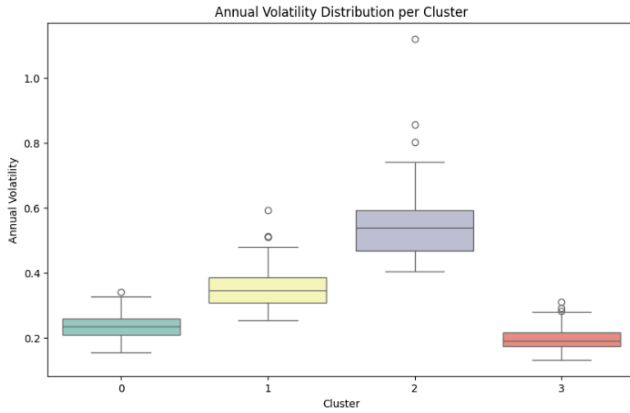
# Add cluster labels to stock_metrics
# stock_metrics['Cluster'] = kmeans_labels # This line is removed as 'Cluster' is already in stock_metrics

# 1. Scatter plot of Beta vs Annual Volatility colored by cluster
plt.figure(figsize=(10,6))
sns.scatterplot(data=stock_metrics, x='Beta', y='Annual_Volatility', hue='Cluster', palette='Set2', s=100)
plt.title('Stock Clusters based on Beta and Annual Volatility')
plt.xlabel('Beta')
plt.ylabel('Annual Volatility')
plt.legend(title='Cluster')
plt.show()

# 2. Boxplots of Annual Volatility by cluster
plt.figure(figsize=(10,6))
sns.boxplot(data=stock_metrics, x='Cluster', y='Annual_Volatility', palette='Set3')
plt.title('Annual Volatility Distribution per Cluster')
plt.xlabel('Cluster')
plt.ylabel('Annual Volatility')
plt.show()

# 3. Boxplots of Beta by cluster
plt.figure(figsize=(10,6))
sns.boxplot(data=stock_metrics, x='Cluster', y='Beta', palette='Set3')
plt.title('Beta Distribution per Cluster')
plt.xlabel('Cluster')
plt.ylabel('Beta')
plt.show()
```

Output of 5b:



References:

1. Panagiotis Chountas CRM week 3 tutorial and lecture notes
2. Panagiotis Chountas CRM week 4 tutorial and lecture notes